

Introducción a Docker: Tecnología y Arquitectura para Ciberseguridad

En el ámbito de la ciberseguridad y el desarrollo moderno, la tecnología de **Docker** se ha convertido en una herramienta fundamental. Docker facilita el uso y manejo de **contenedores**, ofreciendo un método estandarizado, eficiente y altamente portable para empaquetar, distribuir y ejecutar aplicaciones. Este capítulo detallará qué es Docker, sus beneficios clave y su arquitectura interna, esencial para comprender cómo asegurar entornos contenerizados.

¿Qué es Docker? La Tecnología de Contenerización

Docker es la plataforma líder que implementa la **tecnología de contenerización**. A diferencia de las máquinas virtuales (VMs), los contenedores no empaquetan un sistema operativo completo, sino solo la aplicación y todas sus dependencias (librerías, binarios y archivos de configuración). Esto los hace extremadamente livianos y rápidos de iniciar.

Beneficios Clave de la Contenerización

El uso de Docker y contenedores ofrece múltiples ventajas críticas:

1. **Aislamiento (*Isolation*):** Este es quizás el beneficio principal. Los contenedores permiten ejecutar múltiples aplicaciones en un mismo sistema sin que existan conflictos de dependencias entre ellas. Cada contenedor es un proceso aislado con su propio sistema de archivos y espacio de nombres. En el pasado, desplegar varias aplicaciones con distintas dependencias en un mismo servidor era un "caos"; Docker soluciona esto al empaquetar la aplicación con todo lo que necesita.
2. **Eficiencia (*Efficiency*):** Los contenedores son notablemente eficientes porque no son más que **procesos** en el sistema operativo del *host*. No

requieren la carga de un sistema operativo completo dentro de sí mismos, lo que reduce drásticamente el consumo de recursos (CPU, RAM y almacenamiento) en comparación con las máquinas virtuales.

3. **Portabilidad (*Portability*)**: Un desarrollador puede empaquetar su aplicación con todas sus dependencias en una **imagen** de contenedor en su entorno local. Esta imagen puede ser enviada a cualquier servidor *host* y ejecutarse de forma idéntica. La única dependencia necesaria en el servidor es el propio Docker para ejecutar el contenedor.
4. **Estandarización (*Standardization*)**: La popularidad de Docker llevó a la estandarización de esta tecnología. Se crearon estándares que definen cómo deben formarse las imágenes, los contenedores y los registros (repositorios), asegurando que ninguna empresa, incluida Docker, tenga el control total sobre la tecnología.
5. **Escalabilidad (*Scalability*)**: En el contexto de aplicaciones desarrolladas para la nube, los contenedores son ideales para la escalabilidad. Dado que un contenedor es simplemente un proceso, la creación y destrucción de réplicas de una aplicación es extremadamente rápida (milisegundos).
6. **Orquestación (*Orchestration*)**: Docker desarrolló **Docker Swarm**, una herramienta para orquestar contenedores en producción. Aunque posteriormente **Kubernetes** se consolidó como el estándar *de facto* para la orquestación, Docker Swarm fue la respuesta inicial de la compañía a la necesidad de gestionar grandes despliegues de contenedores.
7. **Comunidad y Ecosistema (*Community & Ecosystem*)**: Docker cuenta con una vasta comunidad de usuarios y un ecosistema robusto. Un componente clave es **Docker Hub**, un **registro de imágenes online** donde los usuarios pueden publicar, almacenar y compartir imágenes de contenedores de forma pública o privada.

Mecanismos de Aislamiento en Linux

La capacidad de aislamiento de Docker se basa en dos características fundamentales del **núcleo de Linux (Kernel)**:

- **Namespaces (Espacios de Nombres)**: Esta característica, introducida en 2006, permite asignar a un proceso su **propio espacio de ficheros** o su

propio sistema de ficheros. Es similar al comando `chroot` pero a un nivel más avanzado, garantizando que el proceso solo "vea" los recursos que le han sido asignados.

- **Cgroups (Control Groups - Grupos de Control):** Introducida en 2008, esta característica permite **restringir el uso de recursos** a un proceso o grupo de procesos. Esto significa que se puede limitar la cantidad de CPU, memoria, ancho de banda de red, etc., que un contenedor puede consumir.

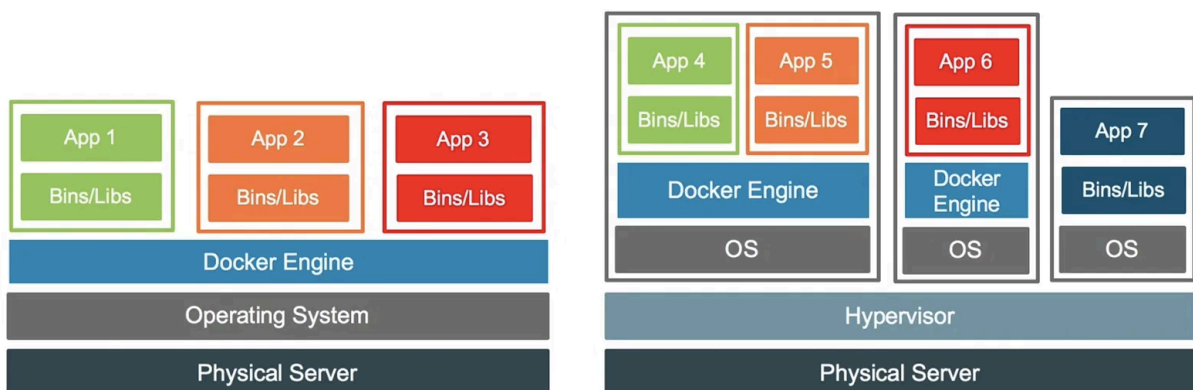
Docker vs. Máquinas Virtuales

Es crucial entender la diferencia entre la contenerización de Docker y la virtualización tradicional con máquinas virtuales.

Representación de Entornos

La siguiente imagen ilustra cómo se despliegan aplicaciones en un entorno de datacenter o VPC utilizando tanto la contenerización como la virtualización.

Your Datacenter or VPC



En el diagrama de la izquierda, se muestra un servidor físico que ejecuta un **Sistema Operativo** y sobre él, el **Docker Engine**. Las aplicaciones **App 1**, **App 2** y **App 3** se ejecutan como contenedores. Estos contenedores solo incluyen la aplicación y sus binarios/librerías (**Bins/Libs**), y todos **comparten el mismo núcleo** del sistema operativo *host*.

En el diagrama de la derecha, se muestra un servidor físico con un **Hipervisor** (el software que gestiona las máquinas virtuales). Sobre el hipervisor, se ejecutan tres **Máquinas Virtuales** (los cuadrados grises). Cada VM tiene su propio **Sistema Operativo (OS)** y su propio núcleo.

- La primera VM tiene instalado el **Docker Engine** y ejecuta dos contenedores (**App 4 y App 5**), los cuales comparten el núcleo de ese sistema operativo dentro de la VM.
- La segunda VM también tiene el **Docker Engine** y ejecuta un contenedor (**App 6**).
- La tercera VM no tiene Docker y ejecuta la **App 7** directamente sobre su Sistema Operativo.

La Diferencia Fundamental

La distinción clave es la siguiente:

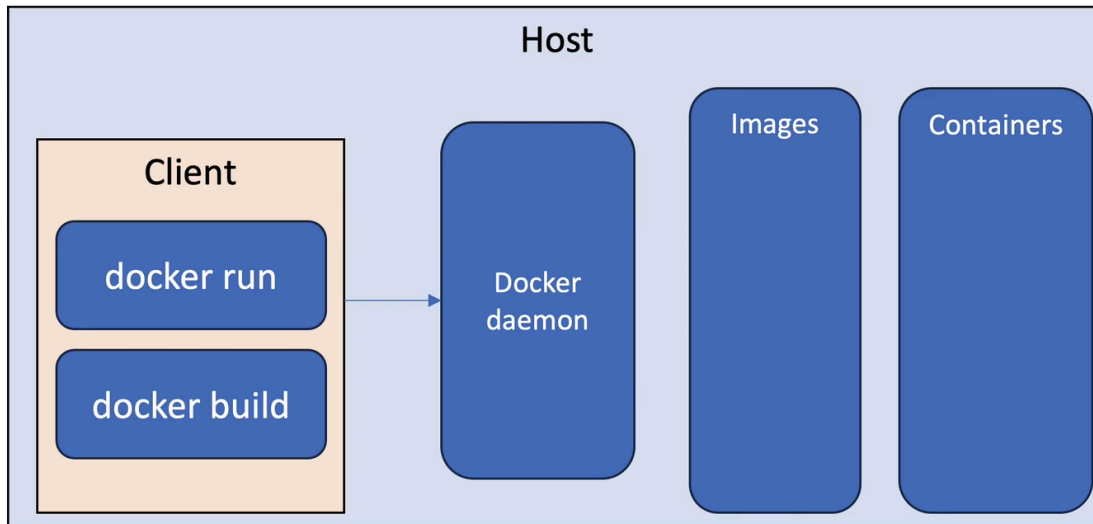
- **Contenedor Docker:** No depende de tener su propio sistema operativo completo. Tiene su propio sistema de ficheros con las **Bins/Libs** necesarias, pero **utiliza y comparte el núcleo** del sistema operativo del *host* (físico o virtual). Es simplemente un proceso aislado.
- **Máquina Virtual (VM):** Cada VM incluye un **Sistema Operativo completo**, incluyendo su propio **núcleo**. Por lo tanto, consume más recursos y tarda más en iniciarse. Los contenedores que se ejecutan dentro de una VM usan el núcleo de ese SO de la VM.

Docker puede ejecutarse indistintamente en una máquina física o virtual, siempre que el sistema operativo subyacente sea Linux (o tenga un núcleo de Linux disponible) para poder hacer uso de las características de **Namespaces** y **Cgroups**.

Arquitectura de Docker

La arquitectura de Docker se basa en un modelo de **Cliente-Servidor**.

Componentes Principales en Linux

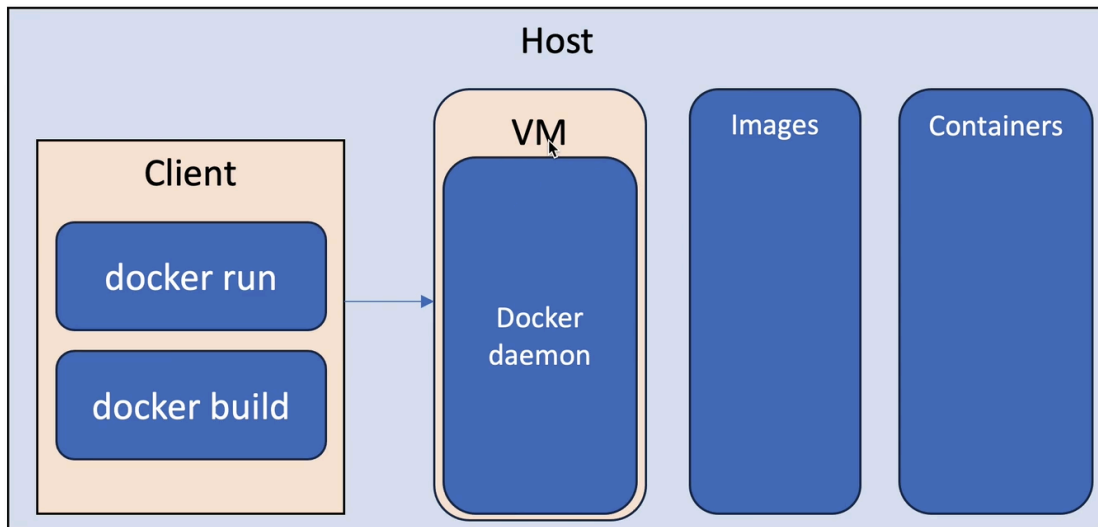


Al instalar Docker en un *host* con una distribución Linux, se instalan principalmente dos componentes:

1. **Docker Daemon (Servidor/Motor):** Es el componente más importante. El *daemon* es un **servicio** de larga ejecución que se ejecuta en el *host* (servidor). Es el responsable de construir, ejecutar y gestionar todos los objetos de Docker (imágenes, contenedores, volúmenes, redes, etc.). Recibe las peticiones del cliente y las procesa.
2. **Cliente Docker:** Es la interfaz principal con el usuario, generalmente a través de la línea de comandos (**docker**). El cliente se comunica con el *daemon* (servidor) mediante una API (normalmente REST). Los comandos como **docker run** o **docker build** son ejecutados por el cliente y enviados al *daemon* para su ejecución.

Otros objetos clave que el *daemon* gestiona y almacena son las **Imágenes** y los **Contenedores**.

Arquitectura en Sistemas No-Linux (Windows/Mac)

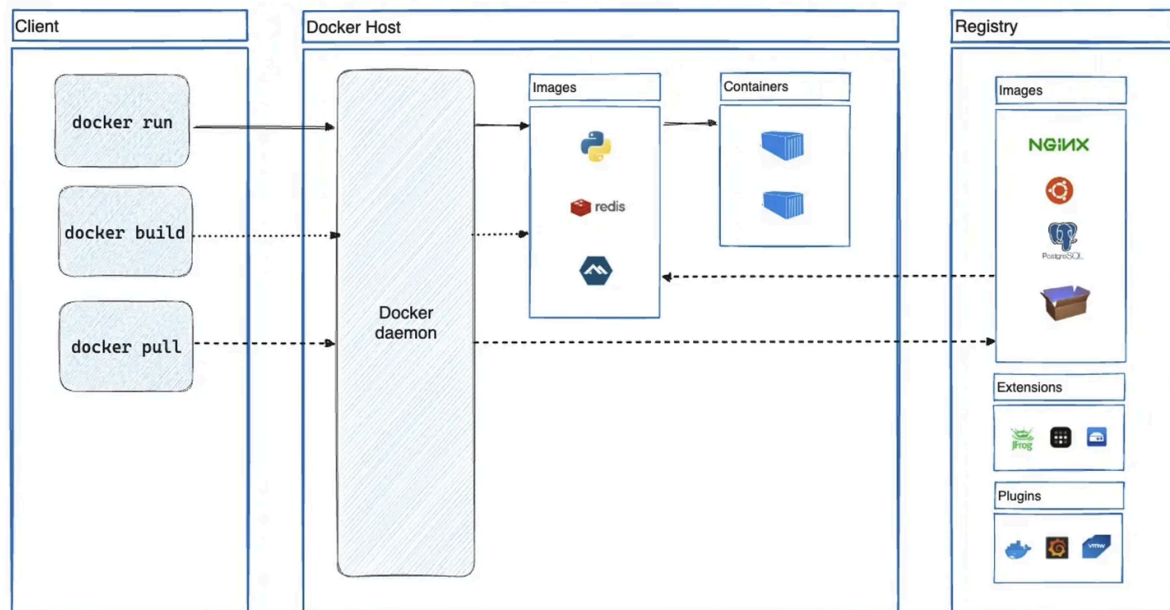


Dado que Docker se basa en las características del núcleo de Linux, los sistemas operativos como Windows o macOS que no usan el núcleo de Linux nativamente requieren un pequeño truco:

1. El software **Docker Desktop** se instala en el *host* (Windows/Mac).
2. Docker Desktop instala una **Máquina Virtual (VM) con Linux** en su interior.
3. El **Docker Daemon** se ejecuta **dentro de esta VM de Linux**.
4. El **Cliente Docker** en el *host* (la línea de comandos o interfaz gráfica) se comunica con el *daemon* que se ejecuta dentro de la VM.

De esta manera, el *daemon* de Docker no se ejecuta directamente en el *host* de Windows o Mac, sino en la VM, permitiendo que los contenedores Linux funcionen correctamente.

El Ecosistema y el Registro (Registry)



La arquitectura se extiende para incluir un tercer componente esencial, el **Registro (Registry)**:

- **Registry:** Es un sistema de almacenamiento y distribución para imágenes de Docker. El más conocido es **Docker Hub**.

La interacción completa entre los componentes se da con comandos clave:

Comando	Ejecutado por el Cliente	Enviado al Daemon	Propósito
docker run	El cliente envía la solicitud al <i>daemon</i> .	El <i>daemon</i> localiza la imagen necesaria (en su almacén o en un Registro), crea un proceso aislado a partir de ella, y lo ejecuta como un Contenedor .	Ejecuta un contenedor a partir de una imagen.

docker build	El cliente envía las instrucciones de construcción (<i>Dockerfile</i>) al <i>daemon</i> .	El <i>daemon</i> utiliza el <i>contexto de construcción</i> para crear una nueva Imagen a partir de las instrucciones y la almacena localmente.	Crea una imagen de Docker a partir de un <i>Dockerfile</i> .
docker pull	El cliente envía la solicitud al <i>daemon</i> .	El <i>daemon</i> se conecta a un Registro (como Docker Hub) y descarga la imagen solicitada para almacenarla localmente.	Descarga una imagen desde un Registro externo.

Manejo de Contextos

El cliente Docker no tiene por qué comunicarse solo con el *daemon* local. Puede comunicarse con **máquinas remotas** usando el concepto de **Contextos**.

- **Contexto:** Un contexto define la conexión entre el cliente y un *daemon* específico.
- **Alojamiento de Recursos:** Todos los contenedores, imágenes y demás recursos de Docker se almacenan y son gestionados por el **Docker Daemon** en la máquina remota o local donde se esté ejecutando.
- **Independencia de Imágenes:** Si se crea una imagen en el **Servidor A**, esa imagen solo existe en el *daemon* de A. No se puede ejecutar directamente en el **Servidor B** porque la imagen no existe allí. Para mover la imagen de A a B, se debe subir la imagen a un **Registro** (e.g., Docker Hub o un registro privado) y luego descargarla (**docker pull**) desde el Servidor B.

Es fundamental tener en cuenta el contexto de trabajo actual para evitar confusiones sobre dónde residen las imágenes y contenedores.

Contenedores Windows

Aunque menos populares, existen los **Contenedores Windows**. Estos se ejecutan como procesos nativos en el sistema operativo Windows, pero **requieren un núcleo**

de Windows para funcionar. Por lo tanto, los contenedores de Windows no pueden ejecutarse directamente en *hosts* Linux, y los contenedores de Linux no pueden ejecutarse directamente en *hosts* Windows (salvo con el truco de la VM de Linux en Docker Desktop).

Conclusiones

En resumen, Docker facilita una forma eficiente de ejecutar aplicaciones:

- Un **Contenedor Docker** es un **proceso** aislado en el *host* que tiene su propia vista del sistema de ficheros (**Namespaces**) y restricciones de recursos (**Cgroups**).
- Docker **no es virtualización**, sino un mecanismo de aislamiento a nivel de proceso que puede ejecutarse sobre máquinas físicas o virtuales.
- El sistema está compuesto por dos componentes principales: el **Servidor (Daemon)**, que gestiona los recursos, y el **Cliente**, que se comunica con el servidor. Este cliente puede ser en modo línea de comandos o interfaz gráfica.
- Los contenedores de Linux requieren un núcleo Linux, mientras que los contenedores de Windows requieren un núcleo Windows.

Configuración e Instalación de Docker para Entornos de Ciberseguridad

Este capítulo detalla el proceso de instalación y configuración de **Docker** en diferentes sistemas operativos, prestando especial atención a las implicaciones arquitectónicas del uso de **Docker Desktop** versus la instalación nativa del motor de Docker. Comprender esta dualidad es crucial para administrar y asegurar correctamente los entornos contenerizados.

Opciones de Instalación y Arquitectura

Docker ofrece diferentes métodos de instalación que dependen del sistema operativo *host* y de si se necesita una Interfaz Gráfica de Usuario (GUI) o solo la Línea de Comandos (CLI).

1. Instalación Nativa (Solo Linux)

- **Sistemas compatibles:** Exclusivamente **Linux**.
- **Arquitectura:** Es la opción nativa ya que Docker hace uso directo de las funcionalidades del **núcleo (kernel) de Linux** (Namespaces y Cgroups).
- **Componentes instalados:** Se instala directamente el **Docker Daemon** (el servicio servidor) y la **utilidad de la línea de comandos (CLI)**, que es el **Cliente Docker**.
- **Ventaja:** Mayor eficiencia al interactuar directamente con el *host* sin la capa de una máquina virtual.
- **Uso comercial:** La instalación nativa de Docker en una máquina Linux es **gratuita** para uso comercial o empresarial.

2. Docker Desktop (Linux, macOS y Windows)

- **Sistemas compatibles:** Linux, macOS y Windows.
- **Arquitectura:** Esta opción siempre incluye una **máquina virtual (VM)** oculta.

1. En **macOS y Windows**, es la **única opción** disponible, ya que estos sistemas operativos no tienen un núcleo Linux nativo ni las propiedades necesarias para Docker.
 2. En **Linux**, es una opción alternativa a la instalación nativa.
- **Funcionamiento:** Docker Desktop instala:
 1. Una **Máquina Virtual con Linux** muy personalizada y mínima.
 2. El **Docker Daemon** se ejecuta **dentro de esta VM**.
 3. El **Cliente Docker** se instala en el *host* y se comunica con el *daemon* que se ejecuta dentro de la VM.
 - **Componentes adicionales:** Además de la CLI, Docker Desktop proporciona una **Interfaz Gráfica (GUI)** que permite interactuar visualmente con la instalación de Docker (ver contenedores, imágenes, etc.).

Consideraciones de Licencia para Docker Desktop

Es vital distinguir entre la instalación nativa y Docker Desktop en términos de coste:

- **Docker Desktop no es gratuito** para **uso comercial** en empresas que cumplan **ambos** criterios: tener más de **250 trabajadores** y/o un ingreso anual superior a **10 millones de dólares**.
- Si una empresa utiliza Docker Desktop (en Linux, macOS o Windows) y supera estos umbrales, debe adquirir una licencia de pago por cada usuario.
- La instalación nativa de Docker en Linux sigue siendo gratuita para uso comercial.

Procedimiento de Instalación de Docker CLI en Ubuntu Linux

El proceso que se muestra a continuación es la instalación del motor y cliente de Docker de forma **nativa** en una distribución Linux (ej. Ubuntu Desktop).

Requisitos y Obtención del Instalador

Antes de instalar, se debe verificar el tipo de CPU (ej. Apple Silicon vs. Intel en Mac) y los requisitos mínimos (ej. 4 GB de RAM y WSL 2 o Hyper-V en Windows). Para Linux, se recomienda la configuración del repositorio de paquetes.

Paso 1: Configurar los Repositorios de Docker

Este paso asegura que se descargue la versión estable y auténtica de Docker.

```
sudo apt-get update
```

```
sudo apt-get install ca-certificates curl
```

1. **Actualizar paquetes e instalar dependencias:** Se actualiza el índice de paquetes e instalan herramientas necesarias como `ca-certificates` (para verificar certificados SSL) y `curl` (para transferir datos).

```
sudo install -m 0755 -d /etc/apt/keyrings
```

2. **Crear el directorio de *keyrings*:** Se crea un directorio seguro para almacenar las claves GPG (GNU Privacy Guard) de Docker.

```
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg  
-o /etc/apt/keyrings/docker.asc
```

```
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

3. **Descargar y asegurar la clave GPG de Docker:** Se descarga la clave pública de Docker y se le asignan permisos de lectura. Esta clave se usa para verificar la firma de los paquetes.

```
echo \
```

```
"deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu \

$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \

sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

4. **Añadir el repositorio de Docker a la lista de fuentes:** Se añade el repositorio de Docker a la lista de fuentes de `apt` para que el sistema sepa dónde buscar los paquetes de instalación.

```
sudo apt-get update
```

5. **Actualizar el índice de paquetes nuevamente:** Se actualiza el índice de paquetes para incluir el nuevo repositorio de Docker.

Paso 2: Instalar el Motor y Cliente de Docker

Se instalan los paquetes principales del motor (`docker-ce`), la CLI (`docker-ce-cli`), el *runtime* de contenedores (`containerd.io`) y *plugins* clave como `buildx` y `compose`.

```
sudo apt-get install docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin
```

Post-Instalación: Gestión de Permisos

Inicialmente, el **Docker Daemon** se ejecuta con privilegios de *root*. Al intentar ejecutar un comando `docker` sin `sudo`, se producirá un error de "Permiso denegado".

Comprobación inicial:

```
docker version
```

La versión del cliente se mostrará, pero la del servidor (daemon) dará: Permiso denegado

```
sudo docker version
```

Con sudo, el acceso al servidor se concede.

Para evitar tener que usar `sudo` en cada comando, Docker crea un grupo de sistema llamado **docker**. Cualquier usuario que pertenezca a este grupo puede ejecutar comandos `docker` directamente.

Procedimiento para añadir el usuario al grupo **docker**:

```
cat /etc/group | grep docker
```

1. **Verificación del Grupo:** El grupo `docker` se crea automáticamente durante la instalación. Se puede verificar su existencia.

```
sudo usermod -aG docker $USER
```

2. **Añadir el usuario actual al grupo `docker`:** El comando `usermod` modifica las propiedades del usuario actual (`$USER`) añadiéndolo (`-aG`) al grupo `docker`.

```
newgrp docker
```

3. **Aplicar el nuevo grupo:** Para que la pertenencia al grupo tenga efecto sin reiniciar, se utiliza el comando `newgrp` para iniciar una nueva sesión de shell con los grupos actualizados, o se puede cerrar la sesión y volver a iniciarla (*logout y login again*).

Comprobación final (sin `sudo`):

```
docker version
```

Ahora ambos, cliente y servidor, se comunican correctamente.

Instalación y Activación de Docker Desktop

Una vez instalado el motor y cliente nativo (opcionalmente), se puede proceder a instalar Docker Desktop para acceder a la GUI y al motor de la máquina virtual (VM).

Paso 1: Descargar el paquete `.deb`

Se descarga el paquete de instalación de Docker Desktop (ej.

`docker-desktop-4.27.2-amd64.deb`) en la carpeta de *Downloads*.

Paso 2: Instalar el paquete con **apt**

Se utiliza **apt-get install** con la ruta local del archivo.

```
cd Downloads
```

```
sudo apt-get install ./docker-desktop-4.27.2-amd64.deb
```

Nota: La documentación oficial de Docker indica que durante esta instalación puede aparecer un error que, en principio, es seguro ignorar.

Paso 3: Arrancar Docker Desktop y Cambios de Contexto

Al iniciar Docker Desktop por primera vez, se deben aceptar los acuerdos de servicio y la licencia.

Comprobación del cambio de contexto:

Antes de arrancar Docker Desktop, el cliente se comunica con el motor nativo instalado en el *host* de la VM de Linux:

```
docker version
```

Servidor: Docker Engine - Community (Motor nativo del host)

Al **arrancar** Docker Desktop, este inicia su máquina virtual interna y cambia automáticamente el **contexto** del cliente Docker. Ahora, el cliente se comunica con el *daemon* que se ejecuta dentro de la VM de Docker Desktop.

```
docker version
```


Servidor: Docker Desktop 4.27.2 (137060) (Motor dentro de la VM de Docker Desktop)

El Problema de la Dualidad de Contextos

Este cambio de contexto es crucial. El motor nativo y el motor de Docker Desktop tienen **espacios de almacenamiento de imágenes y contenedores separados**.

Ejecución del Primer Contenedor (**hello-world**):

Al ejecutar el comando de prueba:

```
docker run hello-world
```

1. El cliente envía la orden al *daemon* de **Docker Desktop**.
2. El *daemon* de Docker Desktop descarga la imagen **hello-world** desde el Docker Hub.
3. El *daemon* ejecuta un contenedor a partir de ella, imprime el mensaje "Hello from Docker!", y el contenedor se detiene (**Exited**).

Verificación de Recursos (Docker Desktop Activo):

```
docker image ls
```

Muestra la imagen 'hello-world', ya que el cliente está apuntando al motor de Docker Desktop.

Comprobación al Apagar Docker Desktop:

Si se **detiene** el servicio de Docker Desktop, el contexto del cliente revierte al **motor nativo** instalado en el *host* de Linux.

Verificación del Contexto:

```
docker version
```

Servidor: Docker Engine - Community (Vuelve a ser el motor nativo del host)

Verificación de Imágenes:

```
docker image ls
```

NO muestra la imagen 'hello-world'

Conclusión Importante: Las imágenes y contenedores creados y ejecutados contra el motor de la máquina virtual de Docker Desktop **no están disponibles** en el contexto del motor nativo de Docker instalado en el sistema operativo, y viceversa. Es esencial saber **con qué contexto se está trabajando** para evitar errores de recursos no encontrados.

Gestión y Comandos de Docker: CLI vs. Docker

Desktop

Una vez instalado Docker, es fundamental familiarizarse con las herramientas de gestión que ofrece. Aunque **Docker Desktop** provee una interfaz gráfica amigable, la **Línea de Comandos (CLI)** es la herramienta primordial en ciberseguridad y automatización, ofreciendo mayor flexibilidad y potencia, especialmente en entornos remotos o en *pipelines* de desarrollo/operaciones (DevSecOps).

I. Introducción al Entorno Docker Desktop (GUI)

Docker Desktop no solo gestiona la máquina virtual subyacente (en Windows/Mac), sino que también proporciona una Interfaz Gráfica de Usuario (GUI) para interactuar con el motor de Docker.

Configuración Global (*Settings*)

Dentro de los ajustes de Docker Desktop, se encuentran opciones vitales para la gestión de recursos y la seguridad del entorno:

1. **Recursos (*Resources*):** Permite limitar el consumo de recursos del motor de Docker que se ejecuta en la máquina virtual. Esto incluye limitar el número de **CPUs**, la **Memoria**, la **Swap** y el espacio de **Disco** que Docker puede utilizar en el *host*.
2. **Uso compartido de Archivos (*File Sharing*):** Esta opción es crítica para la ciberseguridad. Permite decidir **qué directorios del host pueden ser compartidos** con los contenedores mediante **volúmenes**. Esto controla el acceso de los contenedores a los archivos del sistema operativo anfitrión.
3. **Redes (*Networks*) y Proxies:** Configuración de cómo los contenedores acceden a la red externa, incluyendo la especificación de *proxies* para la comunicación si es necesario.

4. **Motor de Docker (*Docker Engine*):** Permite configurar el propio motor de Docker, incluyendo la activación de opciones **experimentales** que pueden ofrecer más funcionalidades pero no son estables.
5. **Constructor (*Builder*):** Opciones para gestionar la construcción de imágenes, incluyendo la posibilidad de crear imágenes en la nube o para distintas arquitecturas.
6. **Kubernetes:** Docker Desktop tiene la funcionalidad de crear un entorno **Kubernetes** local, basado en Docker. Este tema se explorará en detalle al final del módulo.
7. **Extensiones:** Permite añadir funcionalidades extra a Docker Desktop, como *plugins* para el **escaneo de vulnerabilidades** de imágenes, lo cual es fundamental en un máster de ciberseguridad.

Gestión de Contenedores e Imágenes

La interfaz gráfica de Docker Desktop permite una gestión visual de los objetos de Docker dentro del **contexto** activo (el motor dentro de la VM de Docker Desktop).

- **Contenedores:** Se muestra la lista de contenedores. Al hacer clic en uno, se puede acceder a:
 - **Logs:** La salida estándar del proceso del contenedor.
 - **Inspección:** Configuración y metadatos detallados.
 - **Volúmenes:** Información sobre los volúmenes montados.
 - **Estadísticas:** Uso de recursos en tiempo real.
 - **Acciones:** Opciones de control como **Arrancar** (**start**), **Reiniciar** (**restart**), **Parar** (**stop**) y **Borrar** (**rm**).
- **Imágenes:** Se lista el repositorio de imágenes. Al seleccionar una, se accede a:
 - **Información y Construcción:** Detalles sobre cómo se construyó la imagen (a través del **Dockerfile**, que se verá más adelante).
 - **Vulnerabilidades:** Resultados de escaneo (si se activa la extensión) para identificar fallos de seguridad.
 - **Acción:** Opción para **Arrancar** (**run**) la imagen, creando un nuevo contenedor.

La Relación Imagen-Contenedor (Paralelismo con Proceso)

Es vital comprender la diferencia fundamental entre una **Imagen** y un **Contenedor**:

- **Imagen:** Es un fichero (similar a un archivo **tar**) que se almacena en el sistema de ficheros. Es el **fichero binario estático** de una aplicación en el sistema operativo.
- **Contenedor:** Es el **proceso** que se crea en memoria cuando se arranca una Imagen. Docker toma la Imagen, crea un proceso aislado a partir de ella, le asigna su propio sistema de ficheros y aplica las restricciones de recursos (CPU, memoria, etc.).

Analogía: La **Imagen** es al **Contenedor** lo que un **fichero binario ejecutable** es a un **proceso** en el sistema operativo.

II. El Poder de la Línea de Comandos (CLI)

Aunque la GUI es útil, la **Línea de Comandos (CLI)** ofrece la máxima flexibilidad, lo cual es esencial para *scripting*, automatización, y el trabajo en servidores remotos sin interfaz gráfica. Para evitar la confusión de contextos, se recomienda **desactivar Docker Desktop** y trabajar con el motor nativo de Linux (el **Docker Engine - Community**).

Sintaxis General y Grupos de Comandos

El uso general de la CLI de Docker sigue la sintaxis: **docker [OPTIONS] COMMAND**.

Al ejecutar **docker** sin argumentos, se muestra una lista de comandos y sus categorías principales:

1. **Comandos Comunes (Common Commands):** Los comandos básicos de Docker.
2. **Comandos Adicionales:** Opciones menos frecuentes o más específicas.

3. **Comandos de Gestión (*Management Commands*)**: La estructura recomendada y moderna para interactuar con Docker.

La Evolución de la Sintaxis: De Comandos a Grupos de Gestión

Históricamente, los comandos de Docker eran inconsistentes: se usaba `docker ps` (singular/abreviado) para listar contenedores y `docker images` (plural) para listar imágenes, y `docker rmi` para borrar imágenes versus `docker rm` para contenedores.

Para estandarizar y escalar la CLI, Docker introdujo los **Comandos de Gestión**, agrupando las operaciones por el **recurso** sobre el que actúan:

- **`docker container`**: Gestiona contenedores (listar, crear, ejecutar). La operación antigua para listar era `docker ps`, mientras que la moderna recomendada es `docker container ls`.
- **`docker image`**: Gestiona imágenes (descargar, construir, borrar). La operación antigua para listar era `docker images`, mientras que la moderna recomendada es `docker image ls`.
- **`docker volume`**: Gestiona volúmenes (almacenamiento persistente). La operación moderna recomendada para listar es `docker volume ls`.
- **`docker network`**: Gestiona redes de contenedores. La operación moderna recomendada para listar es `docker network ls`.
- **`docker system`**: Gestiona el propio Docker *daemon*. La operación antigua era `docker info`, mientras que la moderna recomendada es `docker system info`.

La sintaxis recomendada es: `docker [RECURSO] [COMANDO]`.

Comandos de Gestión de Recursos Clave

A continuación, se detallan comandos esenciales agrupados por recurso:

Comandos de Gestión de Contenedores (**`docker container`**):

- **run:** Crea y ejecuta un nuevo contenedor a partir de una imagen.
- **ls:** Lista los contenedores en el contexto activo (`docker ps` es un alias compatible).
- **exec:** Ejecuta un comando dentro de un contenedor que ya está en ejecución (fundamental para el *debugging* o acceso de seguridad).
- **logs:** Muestra los logs (salida estándar) de un contenedor.
- **rm:** Elimina uno o más contenedores detenidos.
- **start / stop / restart:** Controla el ciclo de vida de los contenedores.
- **inspect:** Muestra información de bajo nivel (JSON) sobre el contenedor (metadatos, configuración).
- **diff:** Inspecciona los cambios en el sistema de ficheros de un contenedor.

Comandos de Gestión de Imágenes (`docker image`):

- **ls:** Lista las imágenes en el contexto activo (`docker images` es un alias compatible).
- **pull:** Descarga una imagen desde un registro (ej. Docker Hub).
- **push:** Sube una imagen a un registro.
- **build:** Construye una nueva imagen a partir de un **Dockerfile**.
- **rm:** Elimina una o más imágenes.
- **tag:** Etiqueta una imagen de origen con un nombre y/o versión de destino.
- **login:** Inicia sesión en un registro de imágenes (e.g., Docker Hub o un registro privado).
- **version:** Muestra la información de la versión del cliente y del servidor (daemon).

La recomendación es adoptar la sintaxis de gestión (`docker container ls`, `docker image rm`, etc.) ya que proporciona una estructura más lógica y consistente para todas las operaciones en el ecosistema Docker.

Creación y Despliegue de Imágenes Personalizadas con Dockerfile

Para un máster en ciberseguridad, la capacidad de construir imágenes personalizadas es fundamental, ya que permite controlar las dependencias, aplicar parches y configurar servicios de manera segura. Este capítulo detalla el proceso de creación de una imagen Docker personalizada utilizando un **Dockerfile**, partiendo de una imagen base y modificando su contenido.

I. Introducción al Dockerfile: El Plano de la Imagen

Una pieza central y esencial en el ecosistema de Docker para la creación de imágenes es el fichero **Dockerfile**.

¿Qué es un Dockerfile?

El **Dockerfile** es un simple fichero de **texto** que contiene una serie de **instrucciones** que Docker utiliza para construir una imagen de forma automatizada. Estas instrucciones representan los pasos secuenciales que Docker debe seguir durante el proceso de construcción (*build*).

- **Propósito:** Un Dockerfile habilita la **creación reproducible de imágenes portables**. Asegura la consistencia del entorno a través de distintos *hosts* o arquitecturas y facilita el despliegue y la escalabilidad de aplicaciones.
- **Convención de Nombre:** Por defecto, el fichero debe llamarse **Dockerfile** (con la 'D' mayúscula y el resto en minúsculas), sin extensión. Docker lo buscará con este nombre en el directorio de contexto especificado durante el proceso de construcción. Si se usa un nombre diferente, debe indicarse explícitamente con la opción **-f** al comando **docker build**.

Estructura Básica de un Dockerfile

El Dockerfile se compone de una serie de comandos, donde cada comando representa una capa en la imagen final. El ejemplo más simple, como el que vamos a utilizar, muestra la imagen base y una operación de copia:

```
FROM nginx
COPY index.html /usr/share/nginx/html
```

II. Procedimiento Práctico: Creando una Imagen Personalizada de Nginx

El objetivo de este procedimiento es demostrar cómo construir una imagen de un servidor **Nginx** que sirva una página HTML personalizada en lugar de la página de bienvenida por defecto.

Paso 1: Ejecución y Verificación de un Contenedor Base

Antes de personalizar, se comprueba que la imagen base de Nginx funciona correctamente.

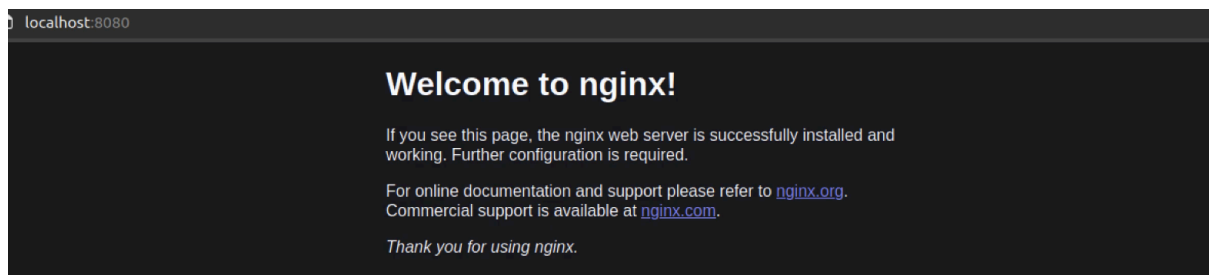
```
docker run -it -p 8080:80 nginx
```

1. **Ejecutar el contenedor Nginx:** Se utiliza el comando `docker run` para descargar la imagen de Nginx (si no está en la caché local) y ejecutar un contenedor, mapeando puertos.

Sintaxis y Propósito:

- **docker run:** El comando para crear y arrancar un contenedor.
- **-it:** Combina **-i** (modo interactivo) y **-t** (asigna un pseudo-TTY), manteniendo la sesión abierta y permitiendo ver la salida del proceso.

- **-p 8080:80: Mapeo de puertos.** Mapea el puerto **8080** del *host* (nuestra máquina local) al puerto **80** del **Contenedor** (donde Nginx está escuchando por defecto). El puerto del *host* puede ser cualquier número entre 1 y 65535.
 - **nginx:** El nombre de la imagen a utilizar (Docker intentará descargar la última versión, **nginx:latest**).
2. **Verificación en el Navegador:** Una vez que el contenedor está en ejecución, se accede a **localhost:8080** en el navegador del *host*. La página de bienvenida por defecto de Nginx confirma que la imagen base es funcional.



3. **Detener el Contenedor:** Se detiene el contenedor en modo interactivo presionando **Ctrl + C** en la terminal, lo que finaliza el proceso de Nginx y, por lo tanto, detiene el contenedor.

Paso 2: Configuración del Entorno de Construcción

Se crea la estructura de archivos necesaria para nuestra imagen personalizada.

1. **Crear un directorio de trabajo.**

```
mkdir first-image
```

```
cd first-image
```

2. **Crear el contenido personalizado (**index.html**):** Se crea el fichero HTML que queremos que sirva nuestro servidor.

```
code . # 0 cualquier editor de texto
```

```
# Crear index.html con el siguiente contenido, por ejemplo:
```

```
# <html><body><h1>Hello World</h1></body></html>
```

3. **Crear el Dockerfile:** Se crea el fichero de instrucciones para Docker, asegurándose de usar el nombre correcto (**Dockerfile** con la D mayúscula).

Fichero **Dockerfile**:

```
Dockerfile
```

```
FROM nginx
```

```
COPY index.html /usr/share/nginx/html
```

- **FROM nginx:** Esta es la instrucción más importante. Define la **imagen base** sobre la que se construirá la nueva imagen. En este caso, se usa la imagen oficial de Nginx. Podríamos haber usado una imagen base mínima como **ubuntu** o **alpine** e instalar Nginx manualmente, pero usar una imagen de Nginx como base es más simple para este propósito.
- **COPY index.html /usr/share/nginx/html:** Esta instrucción copia la fuente (**index.html** en el directorio de contexto) al destino dentro de la imagen. El destino **/usr/share/nginx/html** es la ruta estándar donde Nginx espera encontrar su contenido web por defecto.

Paso 3: Construcción de la Imagen

Se utiliza el comando **docker build** para procesar el Dockerfile y crear la imagen.

1. **Ejecutar el comando de construcción:**

```
docker build -t my-first-image .
```

- **docker build**: El comando para iniciar el proceso de construcción de la imagen.
- **-t my-first-image**: El *flag t* (de *tag*) etiqueta la nueva imagen con un nombre legible, en este caso, **my-first-image**. Sin esta etiqueta, la imagen existiría solo con su ID de *hash*.
- **.**: Este punto final indica el **contexto de construcción**. Le dice a Docker que busque el Dockerfile y cualquier archivo referenciado (como **index.html**) en el **directorio actual**. Este contexto se empaqueta y se envía al Docker Daemon para el proceso de construcción.

2. **Verificar la existencia de la nueva imagen**: Una vez que el *build* finaliza (mostrando los pasos del **FROM** y **COPY**), se comprueba que la imagen está lista.

```
docker image ls
```

- La lista mostrará la imagen base **nginx** y la imagen recién creada, **my-first-image**.

Paso 4: Ejecución y Verificación del Contenedor Personalizado

Finalmente, se arranca un nuevo contenedor basado en la imagen personalizada.

1. **Ejecutar el contenedor personalizado**: Se utiliza el mismo comando **docker run**, pero se reemplaza el nombre de la imagen por el de nuestra imagen recién creada.

```
docker run -it -p 8080:80 my-first-image
```

- Se utiliza el mismo mapeo de puertos (**-p 8080:80**).
- Se especifica la imagen **my-first-image**.

2. **Verificación del Contenido Personalizado:** Al acceder de nuevo a `localhost:8080` en el navegador (o simplemente refrescando la página si el contenedor anterior estaba detenido), se observa el contenido personalizado: **"Hello World"**. Esto confirma que nuestra imagen personalizada se construyó correctamente y el contenedor está sirviendo el contenido modificado.

Conclusión: Hemos demostrado el ciclo completo de creación de imágenes: prueba de la base, definición del **Dockerfile** (instrucciones **FROM** y **COPY**), **construcción** (`docker build`), y **ejecución** del contenedor personalizado (`docker run`). Esto sienta las bases para construir entornos controlados y seguros.

Optimizando la Construcción de Imágenes:

Comandos, Contexto y Exclusiones

En un entorno de ciberseguridad, la eficiencia, el control y el conocimiento de cada capa son cruciales. Este capítulo profundiza en aspectos avanzados del **Dockerfile**, detallando las diferencias entre los comandos de copia, explicando el concepto de **Contexto de Construcción** y presentando la herramienta esencial para el rendimiento y la seguridad: el fichero **.dockerignore**.

I. Comandos de Copia: **COPY** vs. **ADD**

Aunque ambos comandos permiten transferir archivos desde el sistema local del *host* hacia la imagen de Docker, existen diferencias funcionales clave. La sintaxis de ambos comandos es simple:

```
COPY <src> <dest>
```

```
ADD <src> <dest>
```

Donde **<src>** es la fuente (archivo o directorio en el *host*) y **<dest>** es el destino dentro de la imagen.

El Comando **COPY**

- **Propósito:** Permite copiar **ficheros y directorios** desde la máquina local del *host* al sistema de ficheros de la imagen.
- **Comportamiento:** Copia la fuente tal cual, sin procesamiento adicional.
- **Mejor Práctica:** Es el comando **preferido** y más recomendado cuando la intención es simplemente mover archivos de un lugar a otro. Su semántica es clara y predecible.

El Comando **ADD**

El comando **ADD** ofrece la misma funcionalidad que **COPY**, pero con dos capacidades extendidas:

1. Extracción Automática de Archivos Comprimidos:

- Si la fuente (**<src>**) es un archivo **comprimido** (ej. un fichero **.tar** o **.tar.gz**), **ADD** no solo lo copia, sino que también **descomprime y extrae** su contenido directamente en el directorio de destino (**<dest>**) dentro de la imagen.
- **Demostración Práctica:**
 - Si se copia un archivo **assets.tgz** con **COPY assets.tgz /usr/copy/**, el directorio **/usr/copy/** contendrá el fichero **assets.tgz**.
 - Si se copia el mismo archivo con **ADD assets.tgz /usr/add/**, el directorio **/usr/add/** contendrá el **contenido extraído** de **assets.tgz** (los subdirectorios y ficheros internos), no el archivo **.tgz** en sí.

2. Soporte para URLs Remotas:

- **ADD** permite que la fuente (**<src>**) sea una **URL remota**. Docker descarga el fichero de la URL y lo coloca en el destino de la imagen.
- **Comportamiento de Archivos Comprimidos Remotos:** Es importante notar que si el fichero descargado de la URL es un **.tar** o **.tgz**, **ADD** **no extrae su contenido automáticamente**. Lo copia como un fichero simple.

Buenas Prácticas y Recomendaciones

- **Usar **COPY** por defecto:** Si la única necesidad es transferir archivos o directorios locales, utilice **COPY**. Esto evita la extracción automática no deseada si accidentalmente se intenta copiar un archivo **.tar**.
- **Usar **ADD** solo para funcionalidades extendidas:** Use **ADD** solo si se necesita explícitamente la **extracción automática** de un archivo **.tar** local o

la **copia de un recurso remoto** a través de una URL. Si se necesita descargar y extraer un archivo remoto, es una buena práctica de seguridad usar `RUN curl/wget ...` seguido de `tar -xf ...` para tener un control explícito sobre la descarga y la extracción.

II. El Contexto de Construcción (**Build Context**)

El **Contexto de Construcción** es un concepto fundamental para entender el proceso de `docker build` y su impacto en el rendimiento.

Definición del Contexto

- **Qué es:** Es el conjunto de **archivos y directorios** que están disponibles localmente para el proceso de construcción de Docker al momento de crear una imagen.
- **Especificación:** Se especifica como el último parámetro (una ruta) al comando `docker build`.
 - **Ejemplo:** En `docker build -t demo .`, el punto (.) indica que el contexto es el **directorio actual**.
- **Mecanismo de Transferencia:** Cuando se ejecuta `docker build`, Docker **transfiere todo el contenido del directorio de contexto al Demonio de Docker** (el servicio servidor).

Implicaciones de Seguridad y Rendimiento

1. **Impacto en el Rendimiento:** Si el directorio de contexto contiene muchos archivos o archivos muy grandes (como ficheros temporales, *logs*, dependencias o artefactos de construcción), todo este contenido se transfiere innecesariamente al *daemon*.
 - **Ejemplo de Rendimiento:** Si el contexto contiene cuatro archivos temporales de 100MB cada uno, el contexto transferido será de 400MB, lo que ralentiza significativamente la construcción (`build`) y consume recursos del sistema.

2. **Requisito de Copia:** Solo los archivos y directorios que forman parte del **Contexto de Construcción** están disponibles para ser copiados dentro de la imagen mediante las instrucciones **COPY** o **ADD**. Intentar copiar un archivo fuera del contexto resultará en un error.
-

III. Mitigación de Rendimiento: El Fichero **.dockerignore**

El fichero **.dockerignore** es la herramienta estándar para resolver el problema de rendimiento y seguridad del Contexto de Construcción.

¿Qué es y Cómo Funciona?

- **Definición:** Es un fichero de texto simple que Docker utiliza para determinar **qué ficheros y directorios debe ignorar** y, por lo tanto, **no añadir al Contexto de Construcción**.
- **Mecanismo:** Antes de empaquetar el Contexto para enviarlo al *daemon*, Docker lee el archivo **.dockerignore** y excluye los elementos listados.
- **Sintaxis:** Cada línea del fichero especifica un patrón de exclusión. Las líneas en blanco y las líneas que empiezan con **#** (comentarios) son ignoradas.

Reglas y Excepciones

El fichero **.dockerignore** utiliza patrones de comodines (*wildcards*) y reglas de excepción:

1. Exclusión por Patrón:

- **Sintaxis:** ***.json**
- **Efecto:** Ignora cualquier archivo con la extensión **.json**.
- **Sintaxis:** **node_modules/**
- **Efecto:** Excluye completamente el directorio **node_modules** (una práctica común para dependencias de Node.js).

Ejemplo Práctico: Para evitar copiar los archivos temporales grandes que impactan el rendimiento:

`*.tmp`

○

2. Inclusión por Excepción (Anulación):

- **Sintaxis:** `!patron` (se usa el signo de exclamación `!`)
- **Efecto:** Anula las reglas de exclusión anteriores y **fuerza la inclusión** del archivo o patrón especificado en el contexto. Esto es útil si una regla amplia excluye un fichero específico que sí se necesita.
- **Ejemplo Práctico:** Si se ignoran todos los `.tmp` pero se necesita copiar uno específico llamado `current.tmp`:

■ Contenido de `.dockerignore`:

`*.tmp`

`!current.tmp`

○

- Si la instrucción `COPY` en el Dockerfile intenta copiar un archivo excluido (`COPY current.tmp /tmp` cuando `*.tmp` está excluido y `!current.tmp` no está presente), se producirá un error de construcción, ya que el archivo no se encontrará en el contexto.

Buenas Prácticas de `.dockerignore`

- **Principio de Mínimo Privilegio:** Incluir solo los archivos que sean **estrictamente necesarios** para la ejecución de la aplicación en la imagen.
- **Excluir Artefactos:** Ignorar artefactos de construcción (ej. directorios de compilación, archivos `.o`), archivos de registro (`*.log`), archivos temporales y archivos de prueba.
- **Excluir Dependencias del Host:** Si las dependencias de la aplicación (como `node_modules` o entornos virtuales de Python) se van a instalar **dentro de la imagen** durante la construcción (con `RUN apt-get install` o `RUN npm`

`install`), se deben **excluir** del contexto para evitar copiar versiones específicas del *host*.

- **Control de Versiones:** El fichero `.dockerignore` debe ser parte del sistema de **Control de Versiones** (ej. Git) para asegurar que todo el equipo de desarrollo utilice las mismas reglas de exclusión.

I. Comandos Fundamentales para el Entorno

Al construir una imagen, necesitamos establecer dónde residirán los archivos y cómo se instalarán los paquetes.

1. Establecer el Directorio de Trabajo: **WORKDIR**

El comando **WORKDIR** permite establecer el directorio de trabajo dentro de la imagen en construcción.

Sintaxis:

WORKDIR /ruta/al/directorio

- **Propósito:** Define la ruta predeterminada para cualquier instrucción subsiguiente en el Dockerfile (como **RUN**, **COPY**, **CMD**, etc.).
- **Comportamiento:**
 - Si el directorio especificado **no existe**, Docker lo **creará** automáticamente dentro de la imagen.
 - Cualquier instrucción que se ejecute a partir de ese momento se ejecutará de forma **relativa** a este directorio, a menos que se especifique una ruta absoluta.

2. Ejecutar Comandos de Instalación: **RUN**

El comando **RUN** permite ejecutar cualquier comando de shell dentro del contexto de construcción de la imagen.

Sintaxis:

RUN <comando>

- **Propósito:** Se utiliza principalmente para tareas de configuración e instalación de software y dependencias, como instalar paquetes de sistema (**apt install**), compilar código, crear usuarios o ejecutar *scripts*.
- **Comportamiento:** Cada comando **RUN** se ejecuta en una nueva capa de la imagen. Por lo tanto, se debe buscar minimizar el número de capas

innecesarias combinando comandos, aunque para la instalación de dependencias, cada comando `RUN` es fundamental para el proceso de *caching* de Docker.

II. Demostración Práctica: Contenerización de una Aplicación Flask

Vamos a aplicar los comandos `WORKDIR` y `RUN` para contenerizar una aplicación Python simple que utiliza el *framework* **Flask**.

Archivos Iniciales

El directorio de la aplicación (`my-first-app`) contiene:

1. `app.py`: El código de la aplicación Flask que devuelve "Hello, World!" y se ejecuta en el *host* `0.0.0.0` (para ser accesible externamente en el contenedor).
2. `requirements.txt`: Fichero que lista las dependencias necesarias, en este caso, solo `flask`.

Primera Iteración: Usando `ubuntu` como Imagen Base (Malas Prácticas)

Inicialmente, podríamos intentar usar una imagen base genérica como Ubuntu.

Dockerfile (Intento Inicial):

```
FROM ubuntu
```

```
WORKDIR /app
```

```
COPY . .
```

Al construir y ejecutar este contenedor (`docker build -t myapp .` seguido de `docker run -it myapp`), se encuentran tres problemas principales:

1. **Ejecución de la Aplicación:** El contenedor arranca en una *shell* de Ubuntu, no ejecuta automáticamente la aplicación.

2. **Dependencias del Sistema:** El sistema operativo Ubuntu base no tiene **Python** instalado. Si se intenta ejecutar `python3 app.py` se recibirá un error de comando no encontrado.
3. **Dependencias de la Aplicación:** Aunque se instale Python, todavía se necesitaría instalar **Flask** (y posiblemente `pip` como gestor de paquetes).

Para solucionar esto, dentro del contenedor se tendría que ejecutar: `apt update`, luego `apt install python3`, luego instalar `pip`, y finalmente `pip install flask`. Todo este proceso se puede encapsular en la construcción de la imagen.

Segunda Iteración: Usando una Imagen Optimizada y **RUN** (Buenas Prácticas)

La mejor práctica es **utilizar una imagen base que ya contenga las herramientas esenciales** (como Python y `pip`). Esto minimiza la necesidad de comandos **RUN** para la instalación del *runtime* del lenguaje.

Dockerfile (Final):

```
FROM python:3.10

WORKDIR /app

COPY . .

RUN pip install -r requirements.txt
```

Explicación Detallada de los Pasos de Construcción:

1. **FROM python:3.10:** Se utiliza una **imagen oficial de Python** (versión 3.10). Esta imagen ya incluye Python, `pip` y todas las utilidades base necesarias para un entorno Python.
2. **WORKDIR /app:** Se establece el directorio de trabajo a `/app`. A partir de aquí, las operaciones se realizan dentro de esta carpeta.

3. **COPY . .:** Se copian los archivos de la aplicación (`app.py` y `requirements.txt`) desde el directorio de contexto del *host* al directorio de trabajo (`/app`) de la imagen.
4. **RUN pip install -r requirements.txt:** Se utiliza el comando **RUN** para ejecutar el gestor de paquetes `pip` dentro del proceso de construcción de la imagen, instalando **Flask** (la dependencia listada en `requirements.txt`).

Construcción de la Imagen:

```
docker build -t myapp .
```

Durante este proceso, se ejecuta el comando **RUN** para instalar las dependencias, y la imagen resultante (`myapp`) ya contendrá Flask instalado.

Ejecución del Contenedor (Problema Pendiente):

Al ejecutar el contenedor con la nueva imagen (`docker run -it myapp`), el contenedor sigue abriendo una *shell* de Python en lugar de ejecutar la aplicación Flask automáticamente. Esto ocurre porque aún no se ha especificado el **punto de entrada** (el proceso principal que debe ejecutarse). Este aspecto se resuelve con los comandos **ENTRYPOINT** y **CMD**, que serán tratados en el próximo capítulo.

III. Conclusiones y Buenas Prácticas

La construcción de imágenes seguras y eficientes se basa en tres principios clave relacionados con **FROM**, **WORKDIR** y **RUN**.

1. Elección de la Imagen Base (**FROM**):

- **Priorizar imágenes oficiales:** Utilizar una imagen base que ya contenga el *runtime* y las herramientas necesarias (ej. `python:3.10`, `node:lts`, `openjdk:17`). Esto es más eficiente que usar una imagen base genérica (Ubuntu, Alpine) y luego instalar el *runtime* con **RUN**.

- **Seguridad:** Las imágenes base deben mantenerse **actualizadas**, ya que contienen paquetes y librerías que, al igual que cualquier sistema operativo, pueden tener vulnerabilidades. En entornos empresariales grandes, los equipos internos a menudo mantienen y actualizan imágenes base corporativas.

2. Organización con **WORKDIR**:

- Utilizar **WORKDIR** para organizar la aplicación y establecer un punto de referencia claro (**/app** es una convención común). Esto mejora la legibilidad y previene problemas de rutas.

3. Instalación con **RUN**:

- Utilizar **RUN** para instalar todas las **dependencias** y **utilidades** necesarias para que la aplicación funcione correctamente en tiempo de ejecución.

Configuración de Ejecución del Contenedor:

ENTRYPOINT y CMD

Para un control total sobre un contenedor, especialmente en contextos de seguridad y automatización, es fundamental definir explícitamente el proceso principal que se ejecutará al iniciar el contenedor. Los comandos **ENTRYPOINT** y **CMD** del Dockerfile son las herramientas clave para lograr esto, trabajando juntos para definir la instrucción final de ejecución.

I. ENTRYPOINT: La Puerta de Entrada Inmutable

El comando **ENTRYPOINT** define el **ejecutable principal** del contenedor. Es la primera instrucción que se ejecuta al arrancar el contenedor y, generalmente, está diseñada para ser un *script* de configuración o el binario de la aplicación (por ejemplo, `java`, `python`, o el servidor `nginx`).

Sintaxis y Modos de Uso

Modo	Sintaxis	Propósito
Modo Ejecutable (Recomendado)	<code>ENTRYPOINT</code> <code>["exec",</code> <code> "param1",</code> <code> "param2",</code> <code> ...]</code>	Define el ejecutable y sus parámetros fijos en formato JSON array. Los argumentos se pueden extender con <code>CMD</code> o en la línea de comandos. Inmutable.
Modo Shell	<code>ENTRYPOINT</code> <code>exec param1</code> <code>param2</code>	Ejecuta el comando a través de la <i>shell</i> (<code>/bin/sh -c</code>). No es extensible ; los argumentos de <code>CMD</code> no se añaden.

Características Clave del **ENTRYPOINT**

1. **Inmutabilidad:** Una vez que el contenedor se arranca, el **ENTRYPOINT** definido en la imagen es fijo.
2. **Prevalencia:** Si un Dockerfile contiene múltiples **ENTRYPOINT**, solo el **último** definido es el que prevalece.
3. **Sobreescritura en Tiempo de Ejecución:** Es posible sobrescribir el **ENTRYPOINT** de una imagen al ejecutar **docker run** utilizando el *flag* **--entrypoint**. Esto es una técnica de seguridad y *debugging* común, por ejemplo, para ejecutar una *shell* en un contenedor que de otro modo solo ejecuta su aplicación:

```
docker run -it --entrypoint bash mi_imagen
```

II. **CMD:** Argumentos por Defecto y Ejecución Simple

El comando **CMD** tiene un propósito dual, dependiendo de si un **ENTRYPOINT** ha sido definido o no. Proporciona los argumentos por defecto para el **ENTRYPOINT** o el comando principal si no hay **ENTRYPOINT** definido.

Sintaxis y Modos de Uso

Modo	Sintaxis	Propósito
Argumentos para ENTRYPOINT	<code>CMD</code> <code>["param1", "param2", ...]</code>	El modo recomendado cuando hay un <code>ENTRYPOINT</code> definido en modo ejecutable . Estos parámetros se añaden al <code>ENTRYPOINT</code> .
Comando Principal (Modo Ejecutable)	<code>CMD ["exec", "param1", "param2", ...]</code>	Se usa cuando no hay ENTRYPOINT . Define el ejecutable y sus parámetros por defecto.
Modo Shell	<code>CMD exec</code> <code>param1 param2</code>	Se usa cuando no hay ENTRYPOINT . Ejecuta el comando a través de la <i>shell</i> (<code>/bin/sh -c</code>).

Prevalencia y Sobreescritura

- **Prevalencia:** Si un Dockerfile tiene múltiples `CMD` definidos, solo el **último** es el que prevalece.
 - **Sobreescritura por CLI:** Cualquier argumento o comando que se pase al final de la instrucción `docker run` (**parámetros en tiempo de ejecución**) **sobrescribirá completamente** el `CMD` definido en la imagen.
-

III. Interacción entre ENTRYPOINT y CMD

La forma en que Docker ejecuta el contenedor depende de la combinación de ambos comandos. La tabla adjunta resume las reglas de combinación para determinar el **Comando Final** ejecutado:

	No ENTRYPOINT	ENTRYPOINT exe pe1	ENTRYPOINT ["exe", "pe1"]
No CMD	error	/bin/sh -c exe pe1	exe pe1
CMD ["cmd", "pc1"]	cmd pc1	/bin/sh -c exe pe1	exe pe1 cmd pc1
CMD ["pc1", "pc2"]	pc1 pc2	/bin/sh -c exe pe1	exe pe1 pc1 pc2
CMD cmd pc1	/bin/sh -c cmd pc1	/bin/sh -c exe pe1	exe pe1 /bin/sh -c cmd pc1

Reglas de Combinación (Basado en la Tabla)

El **Comando Final** ejecutado por el *daemon* de Docker sigue estas tres reglas principales:

1. Sin **ENTRYPOINT** y Sin **CMD**:

- **Resultado:** Error. Docker no sabe qué proceso arrancar. Sin embargo, este error a menudo no se ve porque las imágenes base (como **ubuntu** o **python**) siempre heredan o definen un **CMD** por defecto.

2. **ENTRYPOINT** en Modo Shell (Columna 2):

- **Comando Final:** **ENTRYPOINT** siempre se ejecuta a través de **/bin/sh -c** y no es extensible.
- **Efecto:** El valor de **CMD** es **ignorado** por completo, y los argumentos en tiempo de ejecución de la CLI también se ignoran como extensiones. El resultado es siempre el comando del **ENTRYPOINT** en modo shell (ej. **/bin/sh -c exe pe1**).

3. **ENTRYPOINT** en Modo Ejecutable (Columna 3 - Recomendado):

- **Comando Final:** El valor de `ENTRYPOINT` (el ejecutable) se **combina** con el valor de `CMD` (los argumentos) para formar un único comando.
- **Ejemplo (Fila 3, Celda Verde):**
 - `ENTRYPOINT ["exe", "pe1"]`
 - `CMD ["pc1", "pc2"]`
 - **Comando Final Ejecutado:** `exe pe1 pc1 pc2`
- **Argumentos de Línea de Comandos:** Si el usuario pasa argumentos en el `docker run`, estos **sustituyen** por completo el `CMD` en la combinación.

4. Solo `CMD` (Columna 1):

- **Comando Final:** Se ejecuta el `CMD` directamente. Si está en **modo ejecutable** (ej. `CMD ["cmd", "pc1"]`), se ejecuta directamente. Si está en **modo shell** (ej. `CMD cmd pc1`), se ejecuta a través de `/bin/sh -c cmd pc1`.

Patrón Común en Imágenes Oficiales (Ejemplo Nginx)

Las imágenes oficiales (como Nginx) a menudo utilizan el patrón `ENTRYPOINT` con un *script wrapper* y `CMD` para los argumentos por defecto:

- **`ENTRYPOINT: ["/docker-entrypoint.sh"]`**
 - *Propósito:* Un *script* de *shell* que realiza tareas iniciales de configuración (ej. sustitución de variables de entorno, comprobación de permisos) y luego ejecuta el proceso principal que se le pasa como argumento.
- **`CMD: ["nginx", "-g", "daemon off;"]`**
 - *Propósito:* Los argumentos por defecto para el *script* de entrada, que en este caso es el comando real del servidor Nginx.
- **Comando Final sin Parámetros:** `/docker-entrypoint.sh nginx -g daemon off;`

Este patrón permite al usuario simplemente pasar un comando al ejecutar `docker run` para **sobrescribir el CMD y ejecutar cualquier binario** (ej. `docker run -it nginx bash`), manteniendo el *script* de configuración del `ENTRYPOINT`.

IV. Solucionando la Ejecución de la Aplicación Flask

Volviendo a nuestra aplicación Flask, el problema de que el contenedor arrancaba en una *shell* de Python se resuelve definiendo el `CMD` para que apunte a la ejecución de la aplicación.

Dockerfile con `CMD` Definido

Para que la aplicación se ejecute automáticamente, se modifica el Dockerfile para que el `CMD` defina el comando Python:

```
FROM python:3.10

WORKDIR /app

COPY . .

RUN pip install -r requirements.txt

CMD ["python3", "app.py"]
```

Procedimiento de Despliegue y Prueba:

1. Construir la Imagen:

```
cd my-first-app/

docker build -t myapp .
```

2. **Ejecutar el Contenedor y Mapear el Puerto:** Dado que la aplicación Flask escucha por el puerto 5000 y solo será accesible desde el *host* si se mapea, se debe usar el *flag* `-p`.

```
docker run -it -p 5000:5000 myapp
```

- **`-p 5000:5000`:** Mapea el puerto 5000 del *host* al puerto 5000 del contenedor.
3. **Verificación:** Al acceder a `localhost:5000` en el navegador, se verifica el **"Hello World"** de la aplicación Flask. Esto confirma que el **Comando Final** ejecutado fue `python3 app.py`, tal como se definió en el **CMD**.

V. Descubrimiento de **ENTRYPOINT** y **CMD** de Imágenes Existentes

Para fines de ciberseguridad, es vital saber cómo se ejecuta un proceso en un contenedor para analizarlo o modificarlo.

- **`docker history`:** Permite ver las instrucciones del Dockerfile que se utilizaron en cada capa de la imagen, revelando las instrucciones **ENTRYPOINT** y **CMD** finales.

```
docker history python:3.10
```

```
docker history nginx
```

- **`docker inspect`:** Ofrece una visión JSON detallada del contenedor o imagen, donde se pueden encontrar los campos **`Config.Entrypoint`** y **`Config.Cmd`**.

Optimización Avanzada de Imágenes Docker: El Modelo de Capas y Caching

Para construir imágenes robustas, eficientes y seguras, especialmente en el ámbito de la ciberseguridad, es imperativo comprender el modelo de capas de Docker. Este modelo no solo determina cómo se estructura el sistema de archivos de la imagen, sino que también dicta la velocidad de construcción y el tamaño final de la imagen.

I. Fundamentos del Sistema de Capas

Una imagen de Docker no es un archivo monolítico, sino que está construida a partir de una **serie de capas apiladas de solo lectura**. Cada capa representa una porción del sistema de archivos y es el resultado de una instrucción en el Dockerfile.

- **Estructura de Solo Lectura:** Cada capa creada es inmutable y de solo lectura. Cuando se ejecuta un contenedor a partir de esta imagen, Docker añade una **capa de escritura** (o capa *writable*) en la parte superior, donde se almacenan todos los cambios realizados durante la vida útil del contenedor.
- **Identificación Única:** Cada capa se identifica mediante un *hash* único, generalmente generado usando **SHA-256**, derivado del contenido de la capa.
- **Compartición y Reutilización:** Una característica fundamental del sistema de capas es su capacidad para ser compartida y reutilizada. Múltiples imágenes pueden basarse en las mismas capas inferiores (como la capa base de Ubuntu o Python), lo que mejora la eficiencia de la compilación y reduce significativamente la sobrecarga de almacenamiento en el *host* o en el registro de imágenes.

El Sistema de Archivos Copy-on-Write (COW)

Docker implementa el modelo de capas gracias a un mecanismo de sistema de archivos llamado **Copy-on-Write (COW)**.

- **Propósito:** El mecanismo COW permite que las capas compartan archivos comunes, reduciendo la redundancia y optimizando el almacenamiento.

- **Funcionamiento:** Cuando se crea por primera vez un archivo o directorio, los datos se comparten entre el original (en una capa inferior) y su nueva copia (en una capa superior o en la capa de escritura del contenedor). Solo cuando se realiza una **operación de escritura** (modificar o eliminar un archivo) sobre la copia, el sistema de archivos detecta el cambio, crea una **nueva copia de los bloques de datos afectados** y escribe los datos modificados en esa nueva ubicación. El archivo original en la capa base permanece inalterado.
- **Implementación:** Docker utiliza sistemas de archivos COW populares como **OverlayFS**, BTRFS o ZFS para gestionar esta superposición eficiente de capas.

II. Estrategia de Caching y Construcción de Imágenes

Durante el proceso de construcción de una imagen, Docker evalúa cada instrucción del Dockerfile de forma **secuencial**. La clave de la optimización reside en cómo Docker utiliza su caché para evitar pasos redundantes.

- **Verificación de la Caché:** Antes de ejecutar una instrucción, Docker compara la instrucción actual y su contexto asociado (ej. los archivos a copiar) con las capas existentes en su caché.
- **Reutilización:** Si la instrucción y sus archivos asociados **no han cambiado** desde la última compilación, Docker **reutiliza** la capa previamente construida (indicado como **CACHED** en el terminal), saltándose la ejecución y acelerando drásticamente el proceso.

Por ejemplo, en la práctica de la aplicación Flask:

```
CACHED [2/4] WORKDIR /app
```

```
CACHED [3/4] COPY . .
```

```
CACHED [4/4] RUN pip install -r requirements.txt
```

- **Invalidación de la Caché:** Si una instrucción en el Dockerfile o el **contexto** (los archivos del directorio actual de construcción) **cambia**, Docker **invalida la caché** para esa capa y para **todas las instrucciones subsecuentes a**

ella. A partir de ese punto de invalidación, todas las capas deben ser reconstruidas desde cero para garantizar que la imagen resultante refleje los cambios.

- **Ejemplo Práctico de Invalidación:** Si en el Dockerfile se usa `COPY . .`, y se crea un nuevo archivo en el directorio actual (ej. `touch temp`), este cambio en el contexto invalida la capa `COPY` y todas las capas que le siguen (`RUN pip install...`).

III. Optimización Estratégica del Dockerfile

El orden de las instrucciones en el Dockerfile es el factor más crítico para aprovechar la caché de capas. Una estrategia de estratificación bien definida puede optimizar el tamaño, la seguridad, el mantenimiento y, crucialmente, el tiempo de construcción de la imagen.

1. Orden de Instrucciones: Separar Código de Dependencias

Un problema común en el desarrollo es que el código fuente (`app.py`) cambia con mucha más frecuencia que las dependencias de la aplicación (`requirements.txt`). Si se usa `COPY . .` antes de `RUN pip install`, cualquier cambio en el código fuente obliga a reinstalar todas las dependencias, lo cual es ineficiente.

Estrategia de Optimización: Colocar las instrucciones que utilizan archivos que cambian con **menor frecuencia** en la **parte superior** del Dockerfile, y las que utilizan archivos que cambian con **mayor frecuencia** en la **parte inferior**.

Dockerfile No Optimizado (Instalación Redundante):

```
FROM python:3.10
```

```
WORKDIR /app
```

```
COPY . . <-- ¡Esta capa se invalida con cada cambio de código!
```

```
RUN pip install -r requirements.txt <-- Esto se ejecuta de nuevo innecesariamente.
```

```
CMD ["python3", "app.py"]
```

Dockerfile Optimizado (Uso Inteligente de Caché):

```
FROM python:3.10
```

```
WORKDIR /app
```

```
# 1. Copiar y ejecutar dependencias (baja frecuencia de cambio)
```

```
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt <-- ¡Esta capa queda en caché!
```

```
# 2. Copiar el código de la aplicación (alta frecuencia de cambio)
```

```
COPY . . <-- Si solo cambia app.py, solo se invalida esta capa.
```

```
CMD ["python3", "app.py"]
```

Efecto Práctico: Al modificar el código (`app.py`) y reconstruir la imagen, las capas

`COPY requirements.txt` y `RUN pip install` se reutilizarán de la caché

(**CACHED**), ya que el archivo **requirements.txt** no ha cambiado. Solo se invalidará y reconstruirá la capa **COPY . .** y las posteriores, minimizando el tiempo de construcción.

2. Agrupación de Comandos **RUN**: Reducción de Capas

Cada instrucción **RUN** crea una **nueva capa** en la imagen. Un exceso de capas (8 capas en un ejemplo básico) puede aumentar la sobrecarga de gestión para el sistema de archivos OverlayFS en el *host* al arrancar el contenedor.

Estrategia de Optimización: Agrupar comandos de instalación o configuración relacionados en una única instrucción **RUN** utilizando el operador de *shell* **&&** (ampersand doble) para encadenar las ejecuciones.

Dockerfile No Optimizado (Exceso de Capas):

```
FROM python:3.10

RUN apt update          <-- Capa 1

RUN apt upgrade -y      <-- Capa 2

RUN apt install curl -y <-- Capa 3

# ... otras 5 capas
```

Resultado: 8 capas totales.

Dockerfile Optimizado (Comandos Agrupados): Se reduce el número de capas al ejecutar los comandos de actualización e instalación en una sola capa:

```
FROM python:3.10

RUN apt update && \

    apt upgrade -y && \
```

```
apt install curl -y  
  
# ... otras 5 capas
```

Resultado: **6 capas** totales, lo que minimiza el impacto en el rendimiento del *host* al usar menos recursos para gestionar las capas.

Consideración Estratégica: No es obligatorio agrupar *todos* los comandos **RUN**. Si un grupo de comandos (como la instalación de un *runtime*) cambia con muy poca frecuencia, es recomendable mantenerlos en una capa separada y colocarlos lo más alto posible en el Dockerfile. Si otro grupo de comandos (como la instalación de un *plugin* secundario) cambia más a menudo, se coloca más abajo.

Regla General de Estratificación:

- **Arriba del Dockerfile:** Instrucciones y archivos que cambian con **poca frecuencia** (ej. Imagen base **FROM**, instalación de librerías de sistema con **RUN apt install**, **COPY requirements.txt**, **RUN pip install**).
- **Abajo del Dockerfile:** Instrucciones y archivos que cambian con **alta frecuencia** (ej. **COPY . .** del código de la aplicación).

NEW